Contenerización Orquestación

Antonio Espín Herranz

Contenidos

- Contenedores Linux vs contenedores Windows
- Multi-stage builds: build → runtime
- Gestión de dependencias con vcpkg en escenarios Docker
- Conceptos clave: Pods, Deployments, Services.
- Despliegue básico de un microservicio en un clúster local (Docker Desktop, Kind o Minikube).
- ConfigMaps y Variables de entorno
- Estrategias de escalado simple
- Ciclos de vida del contenedor y health checks

Introducción

Introducción

Los contenedores permiten empaquetar:

- binarios
- dependencias
- configuración
- Runtime
- En una unidad portable y reproducible.

Contenerización → empaquetar
Orquestación → ejecutar a escala

La orquestación (Kubernetes) gestiona:

- despliegue
- escalado
- networking
- salud
- actualizaciones
- De múltiples contenedores en un clúster.

Introducción

- **¿Qué es un contenedor?**
  - Una unidad ligera que empaqueta:
    - binarios C++
    - dependencias (vcpkg)
    - configuración
    - runtime
    - sistema de archivos aislado
    - **Objetivo:** que la aplicación se ejecute **igual** en cualquier entorno.

- **Ventajas clave:**
  - Reproducibilidad
  - Portabilidad
  - Aislamiento
  - Despliegues rápidos
  - Menos conflictos entre dependencias

Introducción

- **¿Qué es la orquestación?**
- El proceso de gestionar **muchos contenedores** como un sistema coherente.
  - Un orquestador (Kubernetes) se encarga de:
    - despliegue automático
    - escalado horizontal
    - networking entre microservicios
    - health checks y autorecuperación
    - actualizaciones sin downtime
    - distribución de carga
    - **Objetivo:** ejecutar microservicios a escala de forma fiable.

Introducción

- **C++ y vcpkg:**
  - C++ genera binarios nativos → contenedores muy eficientes
  - vcpkg fija versiones y dependencias → builds reproducibles
  - Docker empaqueta todo → despliegues consistentes
  - Kubernetes escala microservicios C++ igual que cualquier otro lenguaje

Contenedores Linux vs contenedores Windows

Contenedores Linux vs Windows

- **Contenedores Linux**
  - Basados en kernel Linux (namespaces + cgroups)
  - Imágenes ligeras (Alpine, Ubuntu, Debian)
  - Ecosistema maduro para microservicios
  - Perfectos para C++ moderno con vcpkg
  - Rápidos, estables y con menor consumo
  - Compatibles con Kubernetes en cualquier entorno

- **Contenedores Windows**
  - Basados en Windows Server Core o Nano Server
  - Imágenes muy pesadas (GBs)
  - Solo útiles para apps **Windows nativas**
    - .NET Framework
    - COM, Win32, IIS
    - C++ con dependencias de Windows
  - Menor rendimiento y mayor consumo
  - Kubernetes solo soporta Windows en nodos dedicados

Conclusión (C++ / vcpkg / microservicios):
- **Linux es la opción recomendada** para microservicios modernos en C++.
- Windows solo cuando dependes de APIs nativas de Windows.

Resumen

Aspecto	Linux	Windows
Kernel	Compartido	Aislado por capa de compatibilidad
Tamaño	Muy ligero	Más pesado
Compatibilidad	Ecosistema completo	Solo apps Windows
Rendimiento	Excelente	Menor
Casos típicos	Microservicios, APIs, runtimes modernos	Apps .NET Framework, legacy

Para microservicios modernos → **Linux.**
Para aplicaciones heredadas (***legacy***) → **Windows**

Prueba

- Probar un contenedor **Linux**:
  - docker run --rm alpine echo "Hola desde Linux"
  - Imagen: 5 mb

- Probar un contenedor **Windows**:
  - docker run --rm mcr.microsoft.com/windows/nanoserver:ltsc2022 cmd /c echo Hola desde Windows
  - El backend se basa en Linux
  - **no matching manifest for linux/amd64**

  - **Este comando falla si estamos en modo Linux, se puede comprobar con:**
    - ***docker info --format "{{.OSType}}"***

Comprobar

- **docker images**

- Si hemos instalado Docker Desktop con Windows Home y con **WSL2** no viene con la opción de cambiar el switch para utilizar contenedores de Windows en vez de Linux.

- Desde **cmd** de Windows
  - Comprobar la versión de Windows con el comando **winver**
  - **El comando: wsl –l -v**

Multi-stage builds
build → runtime

Multi-stage builds (build → runtime)

- Patrón esencial para imágenes pequeñas y reproducibles.

- **Objetivo** Separar el proceso de compilación del entorno de ejecución para obtener imágenes:
  - más pequeñas
  - más seguras
  - más rápidas de desplegar
  - más reproducibles

Multi-stage builds (build → runtime)

- **Estructura típica de las dos etapas:**

- **Stage 1: build**
  - Instala toolchains (g++, clang, cmake…)
  - Instala dependencias (vcpkg)
  - Compila el proyecto
  - Ejecuta tests (opcional)

- **Stage 2: runtime**
  - Copia solo los binarios finales
  - Imagen mínima (Alpine, Debian slim, distroless)
    - Ver DockerHub (imágenes Alpine, Slim, etc.)

  - Sin toolchains ni dependencias de build

Multi-stage builds (build → runtime)

- **Ventajas**:
  - Reduce tamaño de imagen
  - Minimiza superficie de ataque
  - Acelera despliegues
  - Evita “works on my machine”
  - Perfecto para microservicios C++ con vcpkg

- En C++ es especialmente importante separar **build** y **runtime** porque:
  - Las toolchains (gcc, clang, cmake, ninja…) pesan cientos de MB.
  - vcpkg en modo clásico instala muchas dependencias.
  - El runtime real solo necesita:
    - el binario
    - las librerías dinámicas necesarias
    - quizá un archivo de configuración

- Con multi-stage builds:
  - El **primer stage** es un entorno de desarrollo completo.
  - El **segundo stage** es un entorno mínimo para ejecutar el microservicio.

- Esto es exactamente lo que se hace en producción con C++

Imágenes

- Las nuevas imágenes de Microsotf:
  - Microsoft retiró las imágenes antiguas de mcr.microsoft.com/vcpkg/base:*
  - Ahora todo está unificado bajo devcontainers
  - Las nuevas imágenes incluyen:
  - vcpkg
  - toolchains C++
  - CMake moderno
  - usuario no-root
  - entorno de desarrollo completo

- Practica2

Gestión de dependencias con vcpkg en escenarios Docker

Gestión de dependencias con vcpkg en escenarios Docker

- **vcpkg** permite gestionar dependencias **C/C++** de forma reproducible.

- En **Docker**:
  - Se instala en el **stage de build**
  - Se cachea el directorio vcpkg para acelerar builds
  - Se fijan versiones exactas (manifest mode)
  - Se evita incluir toolchains en el runtime

vcpkg en entornos Docker

- **¿Qué aporta vcpkg?**
- vcpkg permite gestionar dependencias C/C++ de forma:
  - reproducible
  - declarativa (manifest mode)
  - portable entre Windows, Linux y macOS
  - integrada con CMake
  - versionada y controlada por proyecto

vcpkg en Docker: enfoque profesional

- **vcpkg se instala en el *stage de build***
  - El contenedor de build contiene toolchains, CMake, vcpkg y compiladores.
  - Todas las dependencias se instalan dentro del contenedor.
  - El entorno del desarrollador no afecta al resultado.

  - ***toolchains****: El conjunto de herramientas necesarias para compilar C/C++: compilador (gcc/g++), enlazador (ld), CMake / Ninja, headers y librerías de desarrollo*
    - ***Todo lo que hace falta para generar el ejecutable.***

Cache de vcpkg para builds rápidos

- **Se cachea el directorio de vcpkg**
  - vcpkg descarga y compila dependencias (a veces muy pesadas).
  - Cachear /usr/local/vcpkg o /root/.cache/vcpkg acelera builds posteriores.
  - Evita recompilar dependencias en cada docker build.
  - *(En la práctica veremos cómo hacerlo con* --mount=type=cache.*)*

Manifest mode : versiones exactas

- **Se fijan versiones exactas con vcpkg.json**
  - Cada proyecto declara sus dependencias.
  - Se fijan versiones concretas.
  - El build es 100% reproducible en cualquier máquina.
  - No depende del entorno del desarrollador.

EJEMPLO

```
{
  "name": "microservicio",
  "version": "1.0.0",
  "dependencies": ["fmt"]
}
```

Modo manifiesto

Forma declarativa para añadir:

- Dependencias
- Versiones
- Configuraciones

Runtime mínimo y seguro

- **El runtime NO incluye vcpkg ni toolchains**
- El stage de runtime solo contiene el binario final.
- No incluye:
  - gcc
  - g++
  - CMake
  - vcpkg
  - headers
  - librerías de desarrollo
- Imagen más pequeña, más segura y más rápida de desplegar.

Flujo completo

Build: entorno completo para compilar (toolchains, vcpkg, CMake).

Runtime: entorno mínimo que solo contiene el binario final.

- **Build** = fabricar.
- **Runtime** = ejecutar.

- **Practica3**

```
+-------------------------------+
| Stage 1: Build                |
| - CMake                       |
| - gcc/g++                     |
| - vcpkg install               |
| - dependencias                |
| - compila el binario          |
+---------------+---------------+
                |
                v
+-------------------------------+
| Stage 2: Runtime              |
| - solo el binario final       |
| - sin toolchains              |
| - imagen mínima               |
+-------------------------------+
```

Desarrollo de microservicios

- **Para desarrollar un microservicio real con Crow + MySQL + dependencias externas, lo correcto es usar EXACTAMENTE este enfoque:**
  - Proyecto CMake
  - Dependencias declaradas en vcpkg.json
  - Compilación dentro de Docker
  - Multi-stage build
  - Runtime mínimo
  - Librerías externas instaladas por vcpkg dentro del contenedor

Desarrollo de microservicios

- Podemos ejecutar microservicios C++ dentro del IDE

- **OJO**, debemos tener las dependencias instaladas en la máquina local (vcpk local).

  - Se necesitan:
    - vcpkg install crow:x64-windows
    - vcpkg install mysql-connector-c:x64-windows

    - Entonces Visual Studio podrá:
      - compilar
      - ejecutar
      - depurar
      - lanzar el microservicio en localhost

Desarrollo de microservicios

- Al utilizar **Docker**:
  - Cuando compilas dentro de Docker:
    - el IDE **ya no ejecuta el binario directamente**
    - el binario existe **solo dentro del contenedor**
    - las dependencias están **solo dentro del contenedor**
    - el runtime está **solo dentro del contenedor**
    - Por eso:
      - **El IDE no puede ejecutar el microservicio directamente**, porque no tiene acceso al entorno del contenedor.
    - Esto es normal y esperado.

Desarrollo de microservicios

- **Desarrollo local (rápido)**
  - Ejecutas desde el IDE
  - Usas vcpkg local
  - Depuras con breakpoints
  - Cambias código rápido

Si, trabajamos en Docker
Desde el minuto 1 NO es práctico
Más lento

- Se desarrolla en local
- Se empaqueta en Docker cuando el microservicio ya funciona

- **Build y ejecución real (reproducible)**
  - Compilas dentro de Docker
  - Ejecutas dentro de Docker
  - El contenedor es el entorno real del microservicio
  - Este es el flujo estándar en backend C++ moderno.

Desarrollo de microservicios

- **Ejemplo**:

```
{
  "name": "microserviciocrow",
  "version": "1.0.0",
  "dependencies": [
    "crow",
    "mysql-connector-c",
    "openssl",
    "fmt"
  ]
}
```

Kubernetes: Pods, Deployments, Services

- Conceptos fundamentales dentro de **K8S**:

- **Pod**
  - Unidad mínima de ejecución.
  - Puede contener uno o varios contenedores que comparten red y volumen.
    - ***Buena práctica 1 ← → 1***

- **Deployment**
  - Controlador que gestiona:
    - réplicas
    - actualizaciones rolling
    - rollback
    - estrategia de despliegue

- **Service**
  - Abstracción de red que expone un conjunto de Pods:
    - ClusterIP
    - NodePort
    - LoadBalancer

- **Idea clave:**
  - **Pod** = ejecución
  - **Deployment** = gestión
  - **Service** = red

Despliegue básico de un microservicio en un clúster local

Despliegue básico de un microservicio en un clúster local

- Podemos usar:
  - Docker Desktop (K8S ya viene integrado)

- Pasos típicos:

  - Construir la imagen Docker

  - Subirla a un registry local o remoto (DockerHub)

  - Crear manifiestos
    - Deployment
    - Service

  - Aplicar con kubectl apply –f

  - Verificar:
    - kubectl get pods

ConfigMaps y Variables de entorno

ConfigMaps y Variables de entorno

ConfigMaps

Para configuración **no sensible**:

- rutas
- flags
- parámetros de servicio

Montables como:

- variables de entorno
- archivos en un volumen

Variables de entorno

Para configuración dinámica:

- URLs
- flags
- toggles

Para secretos → usar **Secrets**, no ConfigMaps.

Estrategias de escalado simple

Estrategias de escalado simple

- **K8S**:
- **Escalado manual**
  - *kubectl scale deployment miapp --replicas=5*
- **Escalado automático (HPA)**
  - Basado en:
    - CPU
    - RAM
    - métricas personalizadas
- **Rolling updates**
  - Actualiza sin downtime.
- **Blue/Green o Canary**
  - Para despliegues controlados.

Ciclos de vida de un contenedor y healt checks

Ciclos de vida de un contenedor y healt checks

- **Liveness probe**
  - Detecta si el contenedor está “vivo”.
  - Si falla → Kubernetes reinicia el Pod.
- **Readiness probe**
  - Indica si el contenedor está listo para recibir tráfico.
  - Si falla → se saca del Service.
- **Startup probe**
  - Para apps que tardan en arrancar.
- **Hooks (preStop, postStart)**
  - Permiten:
    - drenaje de conexiones
    - inicialización
    - Limpieza
- **Idea clave:** los probes son esenciales para resiliencia.